Final Project Jake

Jacob Krol

4/24/26

Introduction

My individual research question is whether a house's value as an asset drives the actual median value of house prices in the Boston area. Using strictly "asset" features (e.g., property tax rate), my experiment contrasts my teammates who are predicting median house value using two other categories of predictors: i) socioeconomic/environmental factors and ii) locational factors.

The data set is sourced from the ISLR2 R package's Boston dataset which, itself, is sourced and modified from the MASS R package.

Here, I will regress median house value onto a subset of the available features which we have manually grouped as "asset value" predictors: property tax rate per $10,000, average number of rooms per home, and age. Age is not the median/average age of a housing unit. Instead, age is the proportion of housing units built before 1940.

To answer the question of whether a house's asset value drives its actual value I will evaluate three regression approaches using root mean squared error (RMSE) as the evaluation metric: i) ridge regresssion, ii) principal component regression, and iii) bagging.

Data

```
library(ISLR2)
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'tibble' was built under R version 4.5.2
```

```
Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'readr' was built under R version 4.5.2

Warning: package 'purrr' was built under R version 4.5.2

Warning: package 'dplyr' was built under R version 4.5.3

Warning: package 'stringr' was built under R version 4.5.2

Warning: package 'lubridate' was built under R version 4.5.2

-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```r
library(glmnet)
```

```
Loading required package: Matrix

Warning: package 'Matrix' was built under R version 4.5.3


Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

    expand, pack, unpack

Loaded glmnet 4.1-10
```

```r
library(pls)
```

```
Attaching package: 'pls'

The following object is masked from 'package:stats':

    loadings
```

```r
library(randomForest)
```

```
randomForest 4.7-1.2
Type rfNews() to see new features/changes/bug fixes.

Attaching package: 'randomForest'

The following object is masked from 'package:dplyr':

    combine

The following object is masked from 'package:ggplot2':

    margin
```

```r
df_bos <- as_tibble(Boston)
df_bos <- df_bos |> select(tax, rm, age, medv)
head(df_bos)
```

```
# A tibble: 6 x 4
    tax    rm   age  medv
  <dbl> <dbl> <dbl> <dbl>
1   296  6.58  65.2  24
2   242  6.42  78.9  21.6
3   242  7.18  61.1  34.7
4   222  7.00  45.8  33.4
5   222  7.15  54.2  36.2
6   222  6.43  58.7  28.7
```

```
summary(df_bos)
```

```
      tax              rm             age              medv      
 Min.   :187.0   Min.   :3.561   Min.   :  2.90   Min.   : 5.00  
 1st Qu.:279.0   1st Qu.:5.886   1st Qu.: 45.02   1st Qu.:17.02  
 Median :330.0   Median :6.208   Median : 77.50   Median :21.20  
 Mean   :408.2   Mean   :6.285   Mean   : 68.57   Mean   :22.53  
 3rd Qu.:666.0   3rd Qu.:6.623   3rd Qu.: 94.08   3rd Qu.:25.00  
 Max.   :711.0   Max.   :8.780   Max.   :100.00   Max.   :50.00  
```

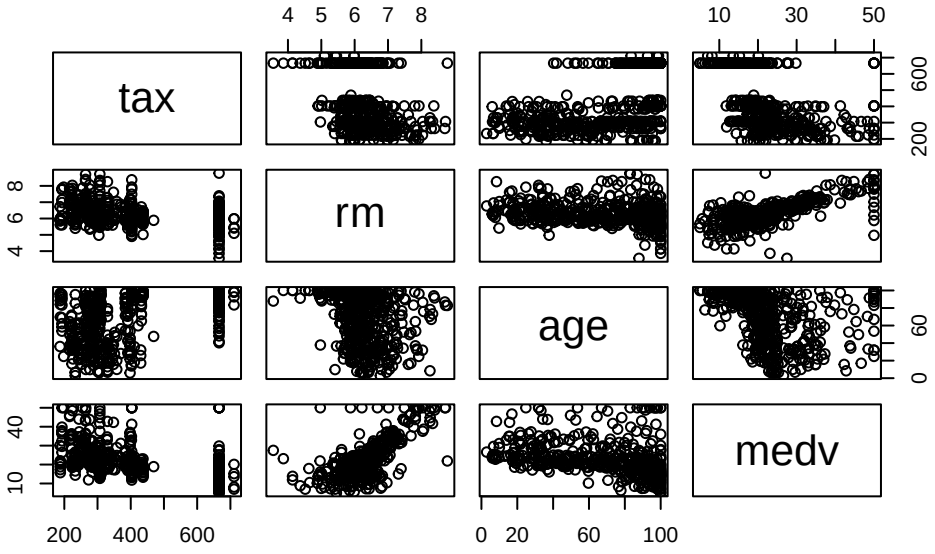
```
# check na values
colSums(is.na(df_bos))
```

```
tax   rm  age medv 
  0    0    0    0 
```

```
# check for duplicates
sum(duplicated(df_bos))
```

```
[1] 0
```

```
pairs(df_bos)
```



Qualitatively, there are associations between median value and the predictors, especially number of rooms.

For evaluation, I will hold out 20% of the data for testing and use the remaining 80% for cross-validation to find optimal hyperparameters for each model.

```
set.seed(0)
train_index <- sample(1:nrow(df_bos), size = 0.8 * nrow(df_bos))
train_data <- df_bos[train_index, ]
test_data <- df_bos[-train_index, ]
```

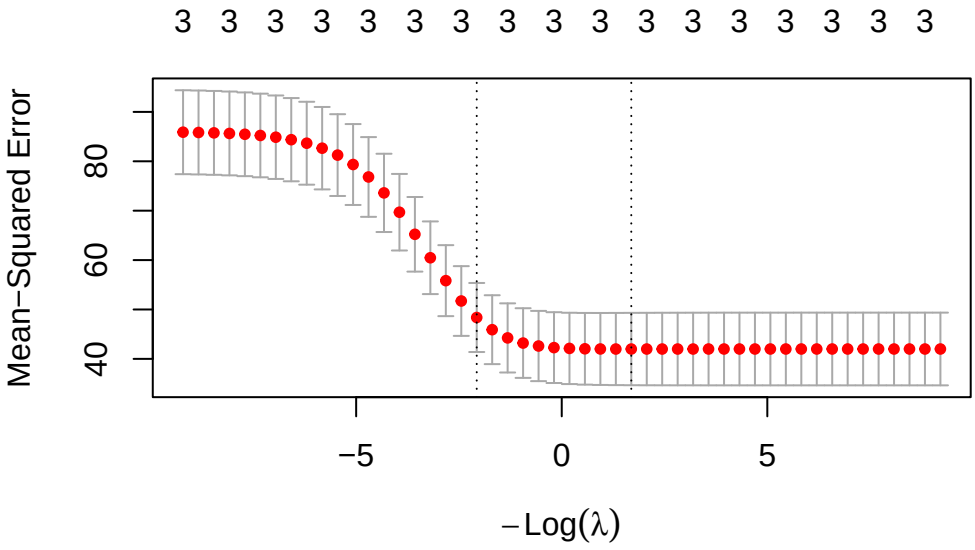
Models

Ridge Regression

For ridge regression, I will select the optimal lambda using 10-fold cross-validation (default in cv.glmnet)on the training set

```
# glmnet requires matrices as input
# and does not use formula syntax
x_train <- as.matrix(train_data |> select(-medv))
y_train <- train_data$medv
x_test <- as.matrix(test_data |> select(-medv))
y_test <- test_data$medv
```

```
# lambdas ranging from 10^4 to 10^-4
lambdas <- 10^seq(4,-4, length = 50)
# alpha=0 is ridge
alpha <- 0
cv_ridge <- cv.glmnet(x_train, y_train, alpha = alpha, lambda = lambdas)
ridge_best_lambda <- round(cv_ridge$lambda.1se, digits=3)
plot(cv_ridge)
```



The best lambda was 7.906.

```
ridge_pred <- predict(cv_ridge, s = ridge_best_lambda, newx = x_test)
ridge_rmse <- round(sqrt(mean((ridge_pred - y_test)^2)), digits=3)
ridge_rmse_scaled <- ridge_rmse * 1000
ridge_rmse_scaled
```

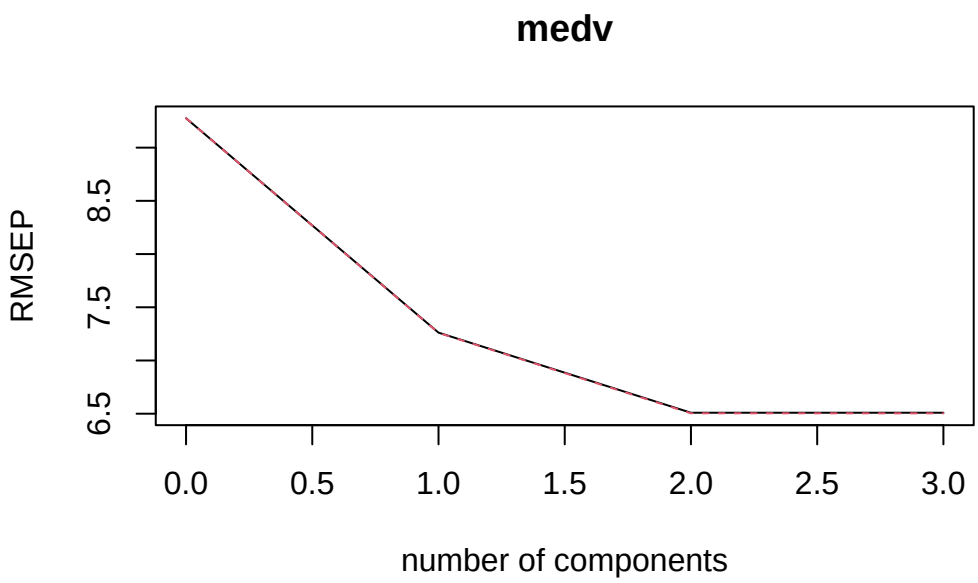
```
[1] 5561
```

The ridge regression model had an RMSE of 5561 dollars.

Principal Component Regression

I will select the optimal number of components using 10-fold cross-validation on the training set.

```
set.seed(0)
pcr_model <- pcr(medv ~ ., data = train_data, scale = TRUE, validation = "CV", segments =
validationplot(pcr_model, val.type = "RMSE")
```



Since 2 and 3 components have similar RMSE, I will choose the smaller model for test set evaluation: 2 components.

```
pcr_pred <- predict(pcr_model, ncomp = 2, newdata = test_data)
pcr_rmse <- round(sqrt(mean((pcr_pred - y_test)^2)), digits=3)
pcr_rmse_scaled <- pcr_rmse * 1000
pcr_rmse_scaled
```

```
[1] 4289
```

The principal component regression model had an RMSE of 4289 dollars.

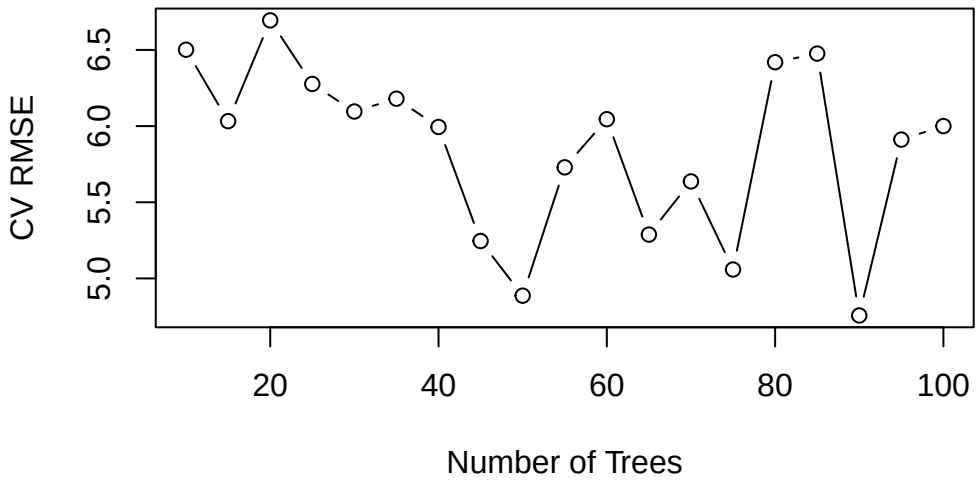
Bagging

For bagging, I will search for optimal tree number using 10-fold cross-validation.

```
set.seed(0)
# for bagging, mtry = number of predictors
mtry <- ncol(train_data) - 1
trees <- seq(10,100, by = 5)
# 10-fold cv
k <- 10
```

```r
tree_rmse <- c()
for (ntree in trees) {
    all_fold_rmse <- c()
    for (i in 1:k) {
        # create folds
        fold_index <- sample(1:nrow(train_data), size = nrow(train_data) / k)
        fold_train <- train_data[-fold_index, ]
        fold_validation <- train_data[fold_index, ]
        # fit model
        bag_model <- randomForest(medv ~ ., data = fold_train, mtry = mtry, ntree = ntree)
        # predict on validation fold
        bag_pred <- predict(bag_model, newdata = fold_validation)
        # calculate RMSE for fold
        fold_rmse <- sqrt(mean((bag_pred - fold_validation$medv)^2))
        all_fold_rmse <- c(all_fold_rmse, fold_rmse)
    }
    # average RMSE across folds for this tree number
    tree_rmse <- c(tree_rmse, mean(all_fold_rmse))
}
```

```r
# plot tree number vs RMSE
plot(trees, tree_rmse, type = "b", xlab = "Number of Trees", ylab = "CV RMSE")
```




```r
best_tree_index <- which.min(tree_rmse)
best_tree <- trees[best_tree_index]
best_tree
```

```
[1] 90
```

The best number of trees was 90.

```
# fit final bagging model with best tree number
set.seed(0)
final_bag_model <- randomForest(medv ~ ., data = train_data, mtry = mtry, ntree = best_tre
bag_pred <- predict(final_bag_model, newdata = test_data)
bag_rmse <- round(sqrt(mean((bag_pred - y_test)^2)), digits=3)
bag_rmse_scaled <- bag_rmse * 1000
bag_rmse_scaled
```

```
[1] 4112
```

The bagging model had an RMSE of 4112 dollars.

Model comparison

```
model_names <- c("Ridge Regression", "Principal Component Regression", "Bagging")
model_rmse <- c(ridge_rmse_scaled, pcr_rmse_scaled, bag_rmse_scaled)
barplot(model_rmse, names.arg = model_names, ylab = "Test RMSE (dollars)", main = "Model C
```